Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question 05/07/2022

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.
Important: Write your name on any additional sheet.

SURNAME (Cognome): .....

NAME (Nome): .....

Matricola: ..... or Person Code: .....

Instructor: ☐ Prof. Breveglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the `Acse` compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the `Lance` language with the ability to count how many bits set to 1 are present in the binary representation of an integer value.

This task is performed by a new operator, called **`count_ones`**, which takes an **arbitrary expression** as its only argument. The value of the operator is the number of bits set to 1 in the binary representation of the value of the expression. The implementation of the operator must perform constant folding whenever possible and must handle negative numbers correctly considering their two's complement representation. Due to the fact that the `Lance` language only supports 32-bit sized integers, the maximum result of `count_ones` must be 32.

The operation and syntax of the `count_ones` operator is shown in the example below. The variable a is initialized with the value 123456, whose binary representation (11110001001000000_2) has 6 digits set to 1. In the first assignment statement, the variable b is assigned with the value of an expression containing `count_ones`. The value of `count_ones` is 6, as its argument is simply the value of a. The rest of the expression adds 100 to this value so, in the end, b is assigned the value 106. In the second assignment, the value of `count_ones` is 13, as the binary representation of the value of the expression $a - 64 + 255$ is 11110001011111111_2 which contains 13 'one' digits. Finally, in the third assignment, the value -1 is represented in binary by 32 'one's, because of two's complement notation. As a result, the value of `count_ones` is 32.

```
int a = 123456, b;

b = count_ones(a) + 100;
/* b == 106 */
b = count_ones(a - 64 + 255);
/* b == 13 */
b = count_ones(-1);
/* b == 32 */
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '−' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..3b64cfc 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID       [a-zA-Z_][a-zA-Z0-9_]*
 "return"          { return RETURN; }
 "read"            { return READ; }
 "write"           { return WRITE; }
+"count_ones"      { return COUNT_ONES; }

 {ID}              { yylval.svalue=strdup(yytext); return IDENTIFIER; }
 {DIGIT}+          { yylval.intval = atoi( yytext );
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..dd7b974 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
 %token RETURN
 %token READ
 %token WRITE
+%token COUNT_ONES

 %token <label> DO
 %token <while_stmt> WHILE
@@ -458,7 +459,57 @@ write_statement : WRITE LPAR exp RPAR
             }
 ;

- exp: NUMBER      { $$ = create_expression ($1, IMMEDIATE); }
+exp: COUNT_ONES LPAR exp RPAR
+   {
+      /* Check if the input expression is immediate or not. */
+      if ($3.expression_type == IMMEDIATE) {
+         /* The expression is immediate, we compute the value at compile time.
+          *  Notice that it is not possible to exploit handle_bin_numeric_op
+          * or handle_binary_operator to handle constant-folding "for free"
+          * because the implementation requires a loop. */
+         int res = 0;
+         unsigned int tmp = $3.value;
+         for (int i=0; i<32; i++) {
+            res += tmp & 1;
+            tmp = tmp >> 1;
+         }
+         /* Assign the expression's semantic value. */
+         $$ = create_expression(res, IMMEDIATE);
+      } else {
+         /* The expression is not immediate, generate code which will compute
+          * the value at runtime.
+          *  Reserve and generate code to initialize the register which will
+          * contain the result. */
+         int r_res = gen_load_immediate(program, 0);
+
+         /* Reserve an auxiliary register, and generate code to initialize it
+          * with the expression's value. */
+         int r_tmp = getNewRegister(program);
```

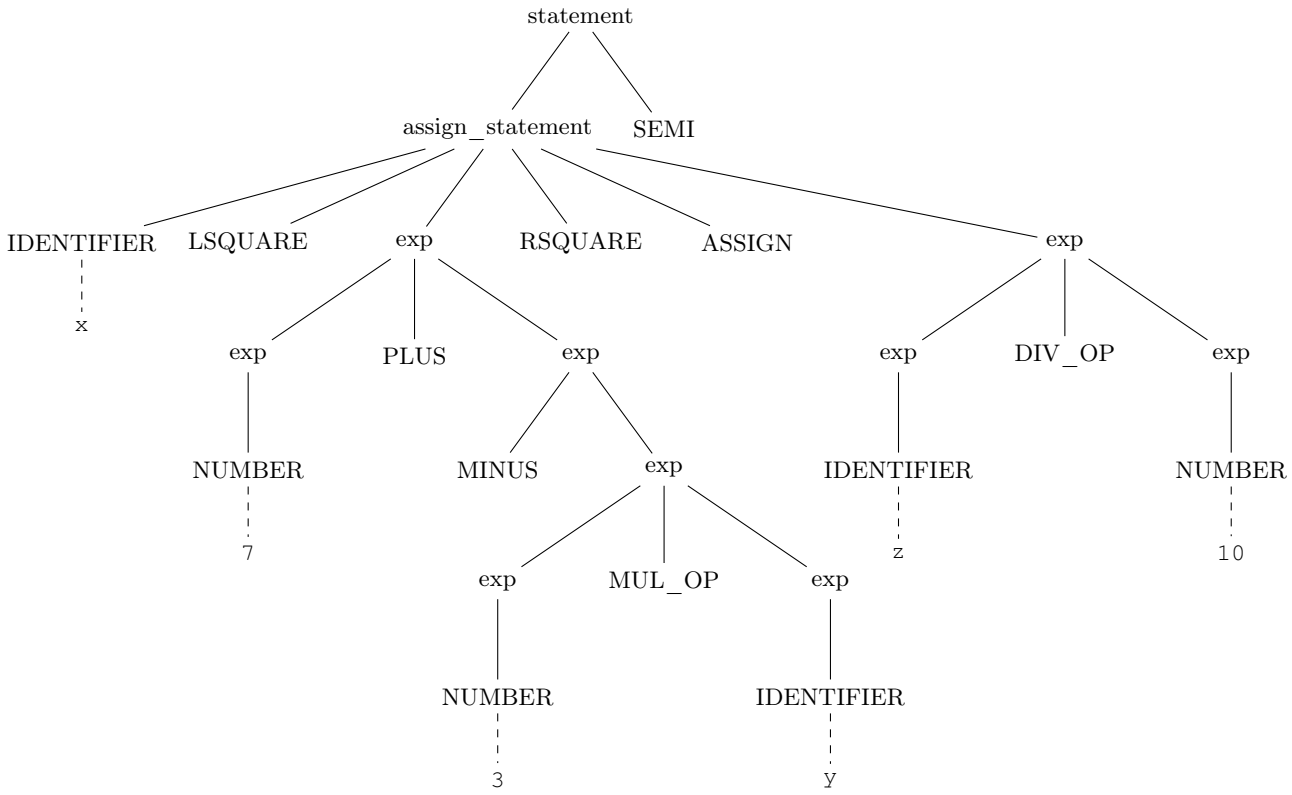
```
+         gen_add_instruction(program, r_tmp, $3.value, REG_0, CG_DIRECT_ALL);
+         /* Reserve and generate code to initialize a register for counting
+          * how many iterations are left in the loop. */
+         int r_i = gen_load_immediate(program, 32);
+
+         /* Generate a label that will point to the beginning of the loop. */
+         t_axe_label *l_loop = assignNewLabel(program);
+         /* Generate code to extract the least significant bit and add it to
+          * the result register. */
+         int r_toAdd = getNewRegister(program);
+         gen_andbi_instruction(program, r_toAdd, r_tmp, 1);
+         gen_add_instruction(program, r_res, r_res, r_toAdd, CG_DIRECT_ALL);
+         /* Generate code to shift out the least significant bit of the
+          * temporary register. */
+         gen_shri_instruction(program, r_tmp, r_tmp, 1);
+         /* Generate code to decrement the loop counter and continue the loop
+          * if it did not yet reach zero. */
+         gen_subi_instruction(program, r_i, r_i, 1);
+         gen_bne_instruction(program, l_loop, 0);
+
+         /* Assign the expression's semantic value. */
+         $$ = create_expression(r_res, REGISTER);
+       }
+   }
+   | NUMBER      { $$ = create_expression ($1, IMMEDIATE); }
    | IDENTIFIER  {
                      int location;

diff --git a/tests/count_ones/count_ones.src b/tests/count_ones/count_ones.src
new file mode 100644
index 0000000..917f442
--- /dev/null
+++ b/tests/count_ones/count_ones.src
@@ -0,0 +1,10 @@
+int a = 123456, b;
+
+b = count_ones(a) + 100;
+write(b); // 106
+
+b = count_ones(a - 64 + 255);
+write(b); // 13
+
+b = count_ones(-1);
+write(b); // 32
```

4. Given the following `Lance` code snippet:

```
x[7 + - 3 * y] = z / 10;
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the* `statement` *nonterminal.* (5 points)

Solution



5. (**Bonus**) Describe how to modify the `count_ones` operator previously implemented to count the bits set to one in the binary representation of the *absolute value* of the argument. In addition, the result of the operator will be positive or negative depending on the sign of the argument. For example, `count_ones(-1)` will have a value of `-1` instead of `32`.